# Can Deep Learning Break a True Random Number Generator?

Yang Yu, *Graduate Student Member, IEEE*, Michail Moraitis, *Graduate Student Member, IEEE*, and Elena Dubrova, *Senior Member, IEEE*

***Abstract*—True Random Number Generators (TRNGs) create a hardware-based, non-deterministic noise that is used for generating keys, initialization vectors, and nonces in a variety of applications requiring cryptographic protection. A compromised TRNG may lead to a system-wide loss of security. In this brief, we show that an attack combining power analysis with bitstream modification is capable of classifying the output bits of a TRNG implemented in FPGAs from a single power measurement. We demonstrate the attack on the example of an open source AIS-20/31 compliant ring oscillator-based TRNG implemented in Xilinx Artix-7 28nm FPGAs. The combined attack opens a new attack vector which makes possible what is not achievable with pure bitstream modification or side-channel analysis.**

***Index Terms*—TRNG, side-channel attack, power analysis, deep learning, FPGA, bitstream modification.**

## I. Introduction

RANDOM numbers are essential to all secure systems. They are used for generating keys, random masks, initialization vectors, nonces, one-time pads, etc [1]. Secure protocols like MACsec, IPsec, or TLS/SSL use random numbers during authentication, key exchange and data streaming. The quality of random numbers greatly affects the security strength of the system [2]. Weak or predictable random numbers produced by a poorly designed random number generator (RNG), or a faulty RNG may open a door into the system in spite of cryptographic protection. The attackers might be able to recover the keys and ultimately take over the system.

A cryptographically secure RNG typically consists of a true random number generator (TRNG), also called *entropy source*, which harvests entropy from some random physical process to generate raw, non-deterministic noise, and a pseudo-random number generator (PRNG), which takes a seed from the TRNG and performs post-processing to extend the seed into a longer sequence with good statistical properties (see Fig. 1).

To provide security assurance, RNGs are extensively tested during their design (to catch design flaws), after production (to find manufacturing defects), and in-field (to detect random faults). Different types of off-line statistical tests, known answer tests, and on-line health tests are used for this purpose.

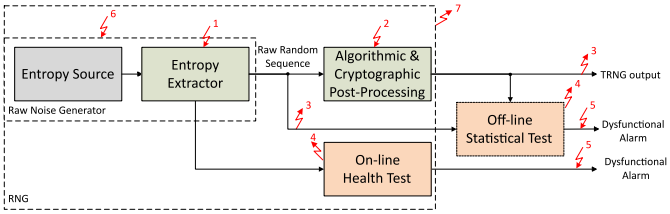



Fig. 1. A block diagram of an AIS-20/31 compliant RNG. The arrows show the attack points of different physical attacks.

*Previous Work:* Previous physical attacks on TRNG implementations can be classified into four types: passive attacks, active attacks, reverse engineering and hardware Trojans.

*Passive attacks* (arrows 3 and 4 in Fig. 1) collect side-channel information from the TRNG implementation and analyze TRNG's activity in order to predict future output bits with a non-negligible probability, e.g., by determining the frequency of a multi-ring oscillator-based TRNG (MURO-TRNG) by electromagnetic (EM) analysis [3]. *Active attacks* (arrows 1, 2 and 5 in Fig. 1) modify the behavior of the TRNG in order to corrupt the TRNG output, e.g., make it biased by fault injection, or switch off the dysfunctional alarm [4]–[6]. *Reverse engineering* (arrow 7 in Fig. 1) aims to locate the TRNG core on chip with the purpose of either mounting an active attack later on, or counterfeiting the chip [7]. *Hardware Trojans* (arrow 6 in Fig. 1) are inserted at the design or manufacturing stage to make functional changes that reduce the entropy of seeds generated by the TRNG [8], or degrade TRNG's performance [9].

*Our Contribution:* This brief presents the first deep learning-based side-channel attack on a TRNG implemented on a field-programmable gate array (FPGA). According to the previous classification, this attack is a combination of reverse engineering with a passive attack. We are inspired by the recent attack on arbiter physical unclonable functions (PUFs) presented in [10] which adds extra flip-flops to the bitstream, through direct bitstream modification, to deduce the PUF responses from less power traces. We mount a similar attack on a MURO-TRNG. Note that the PUF's case is easier because a PUF is a deterministic function that can re-generate the same output response to any given input challenge over and over again (in the error-free case) [11]. Thus, the attacker has a chance to capture multiple power traces representing the computation of the response to a given challenge. So, in the PUF's case, there is no pressure to classify accurately the

Manuscript received February 5, 2021; accepted March 6, 2021. Date of publication March 17, 2021; date of current version April 30, 2021. This work was supported in part by the Swedish Research Council under Grant 2018-04482, and in part by the Vinnova Competence Center for Trustworthy Edge Computing Systems and Applications at KTH Royal Institute of Technology. This brief was recommended by Associate Editor J. Zhang. *(Yang Yu and Michail Moraitis contributed equally to this work.) (Corresponding author: Yang Yu.)*

The authors are with the School of Electrical Engineering and Computer Science, KTH Royal Institute of Technology, 114 28 Stockholm, Sweden (e-mail: yang11, micmor@kth.se; dubrova@kth.se).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TCSII.2021.3066338.

Digital Object Identifier 10.1109/TCSII.2021.3066338

1549-7747 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Cornell University Library. Downloaded on May 24,2021 at 01:44:20 UTC from IEEE Xplore. Restrictions apply.

output bit based on a single power measurement. On the contrary, this is necessary in the TRNG's case since a TRNG is non-deterministic and cannot repeat the same computation twice. Therefore, although the possibility that a similar attack may be applicable to the TRNG context was mentioned in [10], proving that it is possible in practice remained an open problem until now.

The combination of bitstream modification and side-channel analysis opens a new attack vector which is more powerful than each of the two attacks alone. Compared to the former, it does not require any output pins for extracting secrets. So, the attack can be carried out even if all unused output pins are grounded/disabled. In addition, the task of adding extra flip-flops similar to a given one might be easier to automate. More advanced machine learning techniques may potentially enable attacks on FPGA families which are not fully reverse-engineered yet. Some work on applying deep learning for detecting function blocks in bitstreams has already been done [12]. For the latter, it enables extracting secrets from a single power measurement which is not possible with side-channel analysis only.

We demonstrate the attack on the example of an open source AIS-20/31[1] compliant MURO-TRNG [14] implemented in Xilinx Artix-7 28nm FPGAs. We train a multilayer perceptron (MLP) model on one FPGA board implementing the TRNG and then classify the output bits from the same TRNG run on another FPGA board. We evaluate both a case where the output is generated bit-by-bit and a case where the output bits are accumulated in a shift register.

We verified that all the designs, as well as the modified bitstreams derived from them, pass all the NIST SP 800-22 [15] statistical tests. We also verified that they have the same level of entropy as the original TRNG [16] using the NIST SP 800-90B [17] entropy assessment tests. Therefore, the presented attack is capable of bypassing off-line statistical tests, on-line health tests, and countermeasures monitoring the quality of entropy sources at run time, such as [18], [19], making it a stealthier approach compared to active attacks and hardware Trojans.

## II. Background

### A. Multi-Ring Oscillator-Based TRNG

In this brief, we focus on the MURO-TRNG shown in Fig. 2. This design was originally proposed in [16] and further improved in [14]. It uses $r$ ring oscillators (ROs) of identical size. Each RO contains $m + 1$ delay elements (one NAND gate and $m$ buffers). Because the ROs are distinct and independent, their clock phases have a uniform distribution. Since the clock phases are uniformly distributed, the probability that the D flip-flop at the output of the MURO-TRNG will sample a clock edge is the same for all $r$ edges available at the XOR gate.

In [16] the resistance of the MURO-TRNG design to active fault attacks was analyzed. The design was shown to be tolerant to active adversaries to a desired level.

[1]AIS-20/31 stands for application notes and interpretations (AIS) of the scheme number 20/31 which proposes an evaluation methodology for certificated deterministic/physical random number generators used in EU [13].

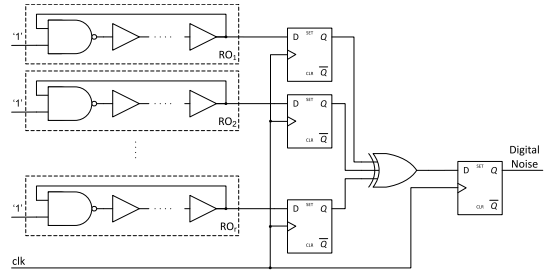



Fig. 2. The MURO-TRNG design.

### B. Profiled Side-Channel Attacks

The main idea of a profiled attack is to create a leakage "profile" of a target device *in advance*, in order to decrease the amount of side-channel information necessary to break the device during the actual attack. This may be important because, for example, physical access to the device under attack is limited, or because the device changes its key frequently.

Profiled side-channel attacks assume that:

1) The attacker has a device, called the *profiling* device, which is similar to the device under attack and runs the same implementation of the target algorithm.
2) The attacker has full control over the profiling device, e.g., knows the secret key, can apply chosen inputs, and measure side-channel signals.
3) The attacker has physical access to the device under attack to measure some side-channel signals during the execution of the target algorithm.

A profiled side-channel attack is performed in two stages: profiling and attack. Profiling can be done by creating a template [20]–[22], or a training model, e.g., an artificial neural network [10], [23]–[25]. To create a profile using the deep learning-based method, a neural network is trained to learn the leakage of the target device for all possible values of the sensitive variable. The training is done using a large number of traces captured from the profiling device(s) which are labeled according to the selected leakage model. At the attack stage, the trained network is used to classify the traces captured from the device under attack.

## III. Adversary Model

We assume that the adversary has physical access to the JTAG or SelectMAP configuration interface and to the bitstream of the device under attack. We also assume that the adversary has equipment for power analysis and expertise in power analysis, deep learning, and bitstream reverse engineering.

a) *Configuration Interface Access:* The access to the SelectMAP or JTAG interface allows the configuration of modified bitstreams. If the FPGA is configured using a microcontroller connected to the FPGA's SelectMAP or JTAG interface, then the attack can be performed through the microcontroller.

b) *Bitstream Access:* The bitstream can be extracted from the device under attack using several methods. One is to wiretap the bus during power-up when the bitstream is transferred from the configuration memory to the FPGA. Another is to directly read out the bitstream from the configuration memory. If a microcontroller is used to configure the FPGA, then the bitstream can be extracted from the microcontroller's firmware.

Authorized licensed use limited to: Cornell University Library. Downloaded on May 24,2021 at 01:44:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I
MLP ARCHITECTURE SUMMARY

| Layer Type | Output Shape | # Parameters |
|---|---|---|
| Dense 1 | (None, 150) | 22650 |
| Dense 2 | (None, 64) | 9664 |
| Dense 3 | (None, 32) | 2080 |
| Output (Dense) | (None, 1) | 33 |

Total Parameters: 34,427

c) *Access Level:* The attacker needs physical access to the device under attack to access a configuration interface and measure the power consumption during the TRNG execution.

Note that there are real scenarios where all the requirements mentioned above are satisfied, e.g., FPGA-as-a-service (FaaS) where the user can keep neither the bitstream nor the data protected from the hardware owner [26], [27].

## IV. EQUIPMENT FOR POWER ANALYSIS

We use ChipWhisperer-Pro to capture power traces for power analysis. The target of the attack is CW305 Artix FPGA board of type XC7A35T-2FTG256.

In our experiments, the FPGA boards were run at 10MHz and sampled at 40MHz, i.e., we took four data points per clock cycle. In the sequel, we refer to the two FPGA devices as $F_1$ and $F_2$. We use $F1$ for profiling and $F2$ as a victim.

## V. NEURAL NETWORK TRAINING

Let $\mathcal{T}_A$ be a set of traces $\mathcal{T}_A$ captured from the device under attack implementing the TRNG and $b_i \in \{0, 1\}$ be the value the output bit generated by the TRNG when $T_i$ is captured, $i \in \{1, \ldots, |\mathcal{T}_A|\}$.

The purpose of training is to create a neural network $\mathcal{N}$ which maps a trace $T_i \in \mathcal{T}_A$ into a *score* vector $P_i = (p_{i,0}, p_{i,1})$, where $p_{i,j}$ is the probability that $b_i = j$, for $j \in \{0, 1\}$, $i \in \{1, \ldots, |\mathcal{T}_A|\}$. The network $\mathcal{N}$ can be viewed as mapping of type $\mathcal{N} : \mathbb{R}^n \to \mathbb{I}^2$ where $n$ is the number of data samples in each $T_i \in \mathcal{T}_A$ and $\mathbb{I} := \{x \in \mathbb{R} \mid 0 \leq x \leq 1\}$.

The neural networks are trained using a set of traces $\mathcal{T}_T$ captured from the profiling device which are labeled by the value of the corresponding TRNG's output bit. We use *binary cross-entropy loss* to quantify the classification error of the network. To minimize the loss, the *gradient* of the loss with respect to the score is computed and back-propagated through the network to tune its internal parameters according to some optimizer. This is repeated for a chosen number of iterations called *epochs*.

Initially, we trained both MLPs and Convolutional Neural Networks (CNNs). Early experiments showed that both types can achieve an equally high classification accuracy in the TRNG's case. Since MLPs are significantly faster to train and traces captured using ChipWisperer are perfectly synchronized, we used MLPs for the rest of our experiments.

After many tests, we have chosen the MLP with the architecture shown in Table I. The input size corresponds to the total number of data samples in the trace, $n = 150$.

## VI. EXPERIMENTAL RESULTS

This section describes the experiments we performed to validate the classification accuracy of the trained MLPs.

The experiments were performed on two versions of the MURO-TRNG design shown in Fig. 2. In the first version, the TRNG generates its output sequence bit-by-bit (**case 1**). In the second version, the TRNG output bits are first accumulated in an 8-bit shift register and then the resulting byte is given to the output (**case 2**). It both cases, a MURO-TRNG with $r = 120$ ROs containing 10 buffers each was used. The MURO-TRNG core is an adaptation of the source code provided by [14].

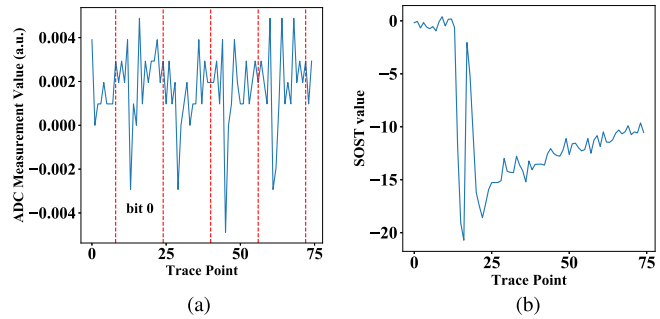



Fig. 3. (a) A MURO-TRNG power trace; (b) t-test for bit 0.

For each implementation of the MURO-TRNG, we captured a set of 500K traces for training, $\mathcal{T}_T$, from the profiling device $F_1$ and recorded the corresponding TRNG bits as labels. Similarly, we captured a smaller set of 10K traces for the attack, $\mathcal{T}_A$, from both $F_1$ and the victim device $F_2$.

### A. Attack Without Bitstream Modification

First, we trained an MLP network on labeled traces captured from the profiling device $F_1$ implementing a MURO-TRNG that generates the output bit-by-bit (case 1) and applied the resulting model to classify traces from $F_2$. The model takes as input the segment of the trace representing a single TRNG output bit. This is repeated for all the bits of the sequence.

The MLP model was able to classify "0"s and "1"s with an accuracy of 0.5489. This is slightly higher than the mean value of the test set 0.5036, however clearly not sufficient for a successful attack. For a successful attack on a TRNG, a close to 100% accuracy based on a single trace is required since the techniques for increasing the signal-to-noise ratio (e.g., repetitive capture) cannot be applied due to the TRNG's non-determinism.

To understand why the attack fails, we analyzed MURO TRNG traces using t-test[2] [28]. The results are shown in Fig. 3. From the distribution of output bit values for the trace point with the maximum sum of squared pairwise t-differences (SOST) shown in Fig. 4(a), we can see that the "0" and "1" cases practically overlap. This justifies the low accuracy of our model.

### B. Attacks With Bitstream Modification

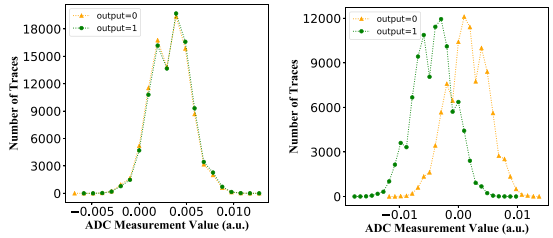
Motivated by [10], we applied a bitstream modification attack to strengthen the TRNG output signal.

The attack consists of the following steps:

1) Extract the bitstream $\mathcal{B}$ from the device under attack.
2) If $\mathcal{B}$ is encrypted, decrypt it using one of the known approaches, e.g., [29]–[33].
3) Make bitstream modifications in $\mathcal{B}$. The result is the modified bitstream $\mathcal{B}^*$.

[2]T-test is a popular statistical technique which is used as a metric for evaluating side-channel leakage and as a tool for feature extraction from side-channel measurements.

Authorized licensed use limited to: Cornell University Library. Downloaded on May 24,2021 at 01:44:20 UTC from IEEE Xplore. Restrictions apply.




(a) N = 0 (b) N = 200

Fig. 4. The distribution of power consumption in implementations with N extra FFs on the trace point with the maximum SOST value (for 500K traces).

4) Load $\mathcal{B}^*$ into the profiling device, capture training traces $\mathcal{T}_T$ and record their labels $b_1, \ldots, b_{|\mathcal{T}_T|}$.
5) Train a neural network $\mathcal{N}$ on labeled traces $\mathcal{T}_T$.
6) Load $\mathcal{B}^*$ back into the device under attack and capture traces $\mathcal{T}_A = (T_1, \ldots, T_{|\mathcal{T}_A|})$.
7) Use $\mathcal{N}$ to classify each trace $T_i \in \mathcal{T}_A$, for all $i \in \{1, \ldots, |\mathcal{T}_A|\}$. The result is the recovered output sequence of bits $b_0, \ldots, b_{|\mathcal{T}_A|}$ generated by the TRNG of the device under attack.

The bitstream modification is a challenging task since even a decrypted bitstream is not easy to meaningfully modify because its format is secret and proprietary. However, proprietary bitstream formats are typically reverse engineered a few years after the introduction of a new series of FPGA. For example, project X-Ray [34] has an extensive database documenting the bitstream format in Xilinx XC7 FPGAs. This enables bitstream modification attacks [35] that can compromise cryptographic algorithm implementations, e.g., AES [36] and SNOW 3G [37].

The aim of step three is to iteratively add extra flip-flops (FFs) to the bitstream until the MLP classification accuracy approximates 100%. In case 1, the extra FFs take as input the output of the FF holding the TRNG output. In case 2, the extra FFs replicate the first bit of the shift register. In both cases, the outputs of the extra FFs are unconnected. To add extra FFs to the bitstream, we used the approach presented in [38].

The bitstream modification process consists of two phases: (1) identification of the target FF which holds the TRNG output bit, and (2) addition of extra FFs to replicate the target.

The identification of the target FF is done through partial bitstream reverse engineering. We start by locating in the bitstream the pin which is used as the output of the TRNG in the design. There, we observe which programmable interconnect points (PIPs) and other FPGA elements (FFs, lookup tables (LUTs), etc) are enabled to backtrack the source of the signal.

The CW305 Artix FPGA board which we use in our experiments uses a custom USB interface for connecting the FPGA device to the computer. So, apart from the TRNG core, the bitstream also implements the USB communication protocol. The FFs implementing the TRNG and the USB protocol have different clock frequencies. To distinguish between them, we check the clock source of each FF visited during the backtracking. When we observe a clock domain swapping, we assume that we left the communication protocol region and entered the TRNG core.

In the TRNG core, we continue backtracking. In case 1, we search for a LUT acting as a multiplexer followed by a FF. This is the target FF which is to be replicated. In case 2, we search for a chain of eight FFs. We replicate the one deepest in the chain, but other choices are possible as well.

TABLE II
THE PROBABILITY TO CORRECTLY CLASSIFY AN OUTPUT BIT OF THE TRNG WITH $k$ EXTRA FFS IN CASE 1. $F_1$ IS USED FOR PROFILING

| $k$ | Device under attack | |
|---|---|---|
| | $F_1$ | $F_2$ |
| 0 | 0.5562 | 0.5489 |
| 25 | 0.7033 | 0.6959 |
| 50 | 0.8180 | 0.8101 |
| 100 | 0.9635 | 0.9562 |
| 200 | 1.0000 | 1.0000 |

TABLE III
THE PROBABILITY TO CORRECTLY CLASSIFY AN OUTPUT BIT OF THE TRNG WITH $k$ EXTRA FFS IN CASE 2. $F_1$ IS USED FOR PROFILING

| $k$ | $F_1$ Target bit position | | | $F_2$ Target bit position | | |
|---|---|---|---|---|---|---|
| | 0 | 4 | 7 | 0 | 4 | 7 |
| 0 | 0.5560 | 0.5469 | 0.5217 | 0.5551 | 0.5383 | 0.5235 |
| 25 | 0.6850 | 0.6926 | 0.6927 | 0.6831 | 0.6967 | 0.6938 |
| 50 | 0.8039 | 0.8196 | 0.8222 | 0.8035 | 0.8187 | 0.8132 |
| 100 | 0.9655 | 0.9802 | 0.9798 | 0.9666 | 0.9786 | 0.9799 |
| 200 | 0.9999 | 0.9999 | 1.0000 | 0.9997 | 1.0000 | 0.9999 |

The second phase is doing the bitstream modification, i.e., adding the extra FFs and routing them. Xilinx XC7 FPGAs have a resource called FDRE which consists of D flip-flops with clock enable (*CE*) and synchronous reset (*SR*). We use these D flip-flops as extra FFs. We add extra FFs one by one by finding unused FDREs in the bitstream and enabling them. We recommended to fully route each FF before starting with the next one because otherwise, routing conflicts may occur.

The new routing is expressed as PIP activations in the bitstream. For each new FF, we connect its *D* input to the *Q* output of the target FF while leaving its *Q* output unconnected. Then we connect its *CE*, *SR* and clock (*C*) inputs to the same sources as the target FF.

### C. Results

This section presents the results of the attacks. Tables II and III show the probability to correctly classify an output bit of the TRNG implementation with $k$ extra FFs using the MLP trained on device $F_1$ for cases 1 and 2, respectively.

We can see from Tables II and III that in both cases the classification accuracy increases as the number of extra FFs grows[3] . With 200 extra FFs, a TRNG output bit can be recovered from a single power trace with an accuracy of almost 100%. In Fig. 4(b) we can see that with 200 extra FFs the distributions corresponding to "0" and "1" become very distinct which justifies the high accuracy of our model.

In the shift register case, all bits except the least significant (LSB) bit 0 require some pre-processing in order to eliminate the effect of shifting of the previous bit. To classify the value of the bit $i$, $i \in \{1, \ldots, 7\}$, we partition the training sets into two groups based on the value of bit position $i - 1$, and then train and classify them individually. In this way, the classification accuracy for bits $i$ can be significantly improved. We can see from Table III that the probability of recovering the whole content of the shift register correctly is at least $0.9997^8 = 0.9976$.

## VII. CONCLUSION

We presented the first side-channel attack on a TRNG FPGA implementation based on deep learning. The attack combines

[3]Note that since the main idea of our attack is to learn the shape of the power traces representing the evaluation of TRNG's output FF, its success probability is not affected by the potential differences in RO distribution (PVT) between the profiling and the attack devices.

Authorized licensed use limited to: Cornell University Library. Downloaded on May 24,2021 at 01:44:20 UTC from IEEE Xplore. Restrictions apply.

deep learning-based power analysis with bitstream modification to deduce the output bits of the TRNG from power traces.

The attack is demonstrated on the example of an open-source AIS-20/31 compliant MURO-TRNG implemented in Xilinx Artix-7 FPGAs. We train an MLP model on one FPGA board implementing the TRNG and then classify the output bits of the same TRNG implementation run on another FPGA board. A close to 100% accuracy can be achieved if 200 extra flip-flops are added to the bitstream of the device under attack.

Since modified TRNG bitstreams have the same level of entropy as the original TRNG implementation and pass all NIST statistical tests, the presented attack is capable of bypassing off-line statistical tests, on-line health tests and countermeasures monitoring the quality of entropy sources at run time.

In future work, we plan to investigate if this attack can be repeated at a several meter distance from a victim device using far-field EM emissions as a side-channel, as in [39].

## REFERENCES


[1] M. Luby, *Pseudorandomness and Cryptographic Applications*. Princeton, NJ, USA: Princeton Univ. Press, 1994.

[2] D. Eastlake, J. Schiller, and S. Crocker, "Randomness requirements for security," IETF, RFC 4086, 2005.

[3] P. Bayon, L. Bossuet, A. Aubert, and V. Fischer, "Electromagnetic analysis on ring oscillator-based true random number generators," in *Proc. IEEE Int. Symp. Circuits Syst.*, Beijing, China, 2013, pp. 1954–1957.

[4] A. T. Markettos and S. W. Moore, "The frequency injection attack on ring-oscillator-based true random number generators," in *Proc. Cryptograph. Hardw. Embedded Syst.*, 2009, pp. 317–331.

[5] P. Bayon *et al.*, "Contactless electromagnetic active attack on ring oscillator based true random number generator," in *Constructive Side-Channel Analysis and Secure Design*. Heidelberg, Germany: Springer, 2012, pp. 151–166.

[6] M. Soucarros, J. Clédière, C. Dumas, and P. Elbaz-Vincent, "Fault analysis and evaluation of a true random number generator embedded in a processor," *J. Electron. Test.*, vol. 29, no. 3, pp. 367–381, Jun. 2013.

[7] U. Guin, K. Huang, D. DiMase, J. M. Carulli, M. Tehranipoor, and Y. Makris, "Counterfeit integrated circuits: A rising threat in the global semiconductor supply chain," *Proc. IEEE*, vol. 102, no. 8, pp. 1207–1228, Aug. 2014.

[8] G. T. Becker, F. Regazzoni, C. Paar, and W. P. Burleson, "Stealthy dopant-level hardware Trojans," in *Proc. Cryptograph. Hardw. Embedded Syst.*, 2013, pp. 197–214.

[9] S. Ghandali, D. Holcomb, and C. Paar, "Temperature-based hardware Trojan for ring-oscillator-based TRNGs," 2019. [Online]. Available: arXiv:1910.00735.

[10] Y. Yu, M. Moraitis, and E. Dubrova, "Why deep learning makes it difficult to keep secrets in FPGAs," in *Proc. DYn. Novel Adv. Mach. Learn. Intell. Cyber Security (DYNAMICS) Workshop*, 2020.

[11] D. Lim, J. W. Lee, B. Gassend, G. E. Suh, M. van Dijk, and S. Devadas, "Extracting secret keys from integrated circuits," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 13, no. 10, pp. 1200–1205, Oct. 2005.

[12] M. Chen and P. Liu, "Deep learning-based FPGA function block detection method using an image-coded representation of bitstream," 2020. [Online]. Available: arXiv:2007.11434.

[13] W. Killmann and W. Schindler, "A proposal for: Functionality classes for random number generators," version 2.0, Rep., 2011. [Online]. Available: https://www.bsi.bund.de/EN/Home/home_node.htm

[14] O. Petura, U. Mureddu, N. Bochard, V. Fischer, and L. Bossuet, "A survey of AIS-20/31 compliant TRNG cores suitable for FPGA devices," in *Proc. 26th Int. Conf. Field Programm. Logic Appl. (FPL)*, Lausanne, Switzerland, 2016, pp. 1–10.

[15] A. Rukhin and L. E. Bassham, "A statistical test suite for random and pseudorandom number generators for cryptographic applications," in *NIST Special Publication 800-22*, vol. 800. NIST: Gaithersburg, MD, USA, May 2001, p. 163.

[16] B. Sunar, W. J. Martin, and D. R. Stinson, "A provably secure true random number generator with built-in tolerance to active attacks," *IEEE Trans. Comput.*, vol. 56, no. 1, pp. 109–119, Jan. 2007.

[17] M. S. Turan, E. Barker, J. Kelsey, K. A. McKay, and M. L. Baish, and M. Boyle, *Recommendation for the Entropy Sources Used for Random Bit Generation*, vol. 800. Gaithersburg, MD, USA: NIST Spec. Publ., 2018.

[18] B. Yang, V. Rožić, N. Mentens, W. Dehaene, and I. Verbauwhede, "TOTAL: TRNG on-the-fly testing for attack detection using lightweight hardware," in *Proc. Design Autom. Test Eur. Conf. Exhibit. (DATE)*, Dresden, Germany, 2016, pp. 127–132.

[19] G. Gimenez, A. Cherkaoui, R. Frisch, and L. Fesquet, "Self-timed ring based true random number generator: Threat model and countermeasures," in *Proc. IEEE 2nd Int. Verif. Security Workshop (IVSW)*, Thessaloniki, Greece, 2017, pp. 31–38.

[20] C. Archambeau, E. Peeters, F.-X. Standaert, and J.-J. Quisquater, "Template attacks in principal subspaces," in *Cryptographic Hardware and Embedded Systems*. Heidelberg, Germany: Springer, 2006, pp. 1–14.

[21] C. Hoffman, C. Gebotys, D. F. Aranha, M. Cortes, and G. Araújo, "Circumventing uniqueness of XOR arbiter PUFs," in *Proc. 22nd Euromicro Conf. Digit. Syst. Design*, Kallithea, Greece, Aug. 2019, pp. 222–229.

[22] G. Camurati, S. Poeplau, M. Muench, T. Hayes, and A. Francillon, "Screaming channels: When electromagnetic side channels meet radio transceivers," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2018, pp. 163–177.

[23] H. Maghrebi, T. Portigliatti, and E. Prouff, "Breaking cryptographic implementations using deep learning techniques," in *Proc. Int. Conf. Security Privacy Appl. Cryptogr. Eng.*, 2016, pp. 3–26.

[24] J. Kim, S. Picek, A. Heuser, S. Bhasin, and A. Hanjalic, "Make some noise. Unleashing the power of convolutional neural networks for profiled side-channel analysis," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, no. 3, pp. 148–179, 2019.

[25] E. Cagli, C. Dumas, and E. Prouff, "Convolutional neural networks with data augmentation against jitter-based countermeasures," in *Cryptographic Hardware and Embedded Systems (CHES)*. Cham, Switzerland: Springer, 2017, pp. 45–68.

[26] H. Englund and N. Lindskog, "Secure acceleration on cloud-based FPGAs—FPGA enclaves," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, New Orleans, LA, USA, 2020, pp. 119–122.

[27] *Amazon EC2 F1 Instances*, AWS, Seattle, WA, USA, Apr. 2017. [Online]. Available:https://aws.amazon.com/ec2/instance-types/f1/

[28] G. Goodwill, B. Jun, J. Jaffe, and P. Rohatgi, "A testing methodology for side-channel resistance validation," in *Proc. NIST Non-Invasive Attack Test. Workshop*, vol. 7, 2011, pp. 115–136.

[29] A. Moradi, D. Oswald, C. Paar, and P. Swierczynski, "Side-channel attacks on the bitstream encryption mechanism of Altera Stratix II: Facilitating black-box analysis using software reverse-engineering," in *Proc. ACM/SIGDA Int. Symp. FPGAs*, 2013, pp. 91–100.

[30] A. Moradi and T. Schneider, "Improved side-channel analysis attacks on Xilinx bitstream encryption of 5, 6, and 7 series," in *Proc. Int. Workshop Construct. Side-Channel Anal. Secure Design*, 2016, pp. 71–87.

[31] S. Tajik, H. Lohrke, J.-P. Seifert, and C. Boit, "On the power of optical contactless probing: Attacking bitstream encryption of FPGAs," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2017, pp. 1661–1674.

[32] M. Ender, A. Moradi, and C. Paar, "The unpatchable silicon: A full break of the bitstream encryption of Xilinx 7-series FPGAs," in *Proc. USENIX Security Symp.*, 2020, pp. 1803–1819.

[33] H. Lohrke, S. Tajik, T. Krachenfels, C. Boit, and J.-P. Seifert, "Key extraction using thermal laser stimulation," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, no. 3, pp. 573–595, 2018.

[34] *Project X-Ray*, SymbiFlow, 2020. [Online]. Available: https://prjxray.readthedocs.io/en/latest/

[35] R. S. Chakraborty, I. Saha, A. Palchaudhuri, and G. K. Naik, "Hardware Trojan insertion by direct modification of FPGA configuration bitstream," *IEEE Design Test*, vol. 30, no. 2, pp. 45–54, Apr. 2013.

[36] P. Swierczynski, M. Fyrbiak, P. Koppe, and C. Paar, "FPGA Trojans through detecting and weakening of cryptographic primitives," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 8, pp. 1236–1249, Aug. 2015.

[37] M. Moraitis and E. Dubrova, "Bitstream modification attack on SNOW 3G," in *Proc. Design Autom. Test Eur. Conf. Exhibit. (DATE)*, 2020, pp. 1275–1278.

[38] M. Moraitis and E. Dubrova, "FPGA bitstream modification with interconnect in mind," in *Proc. 9th Int. Workshop Hardw. Archit. Support Security Privacy (HASP)*, 2020.

[39] R. Wang, H. Wang, and E. Dubrova, "Far field EM side-channel attack on AES using deep learning," in *Proc. 4th ACM Workshop Attacks Solutions Hardw. Security (ASHES)*, Nov. 2020, pp. 35–44.


Authorized licensed use limited to: Cornell University Library. Downloaded on May 24,2021 at 01:44:20 UTC from IEEE Xplore. Restrictions apply.